Project 2- Detect AI Generated Text

Supriyo Sadhya

Introduction

AI generated text is largely prevalent nowadays, especially with the popularity of ChatGPT. The use of AI generated text has become very common especially in the field of academia. Hence, the detection of AI generated text has becoming increasingly important.

Dataset

In this project I use two datasets from Kaggle which contains both human written essays and AI generated texts in response to certain queries.

train_essay.csv - https://www.kaggle.com/competitions/llm-detect-ai-generated-text/data

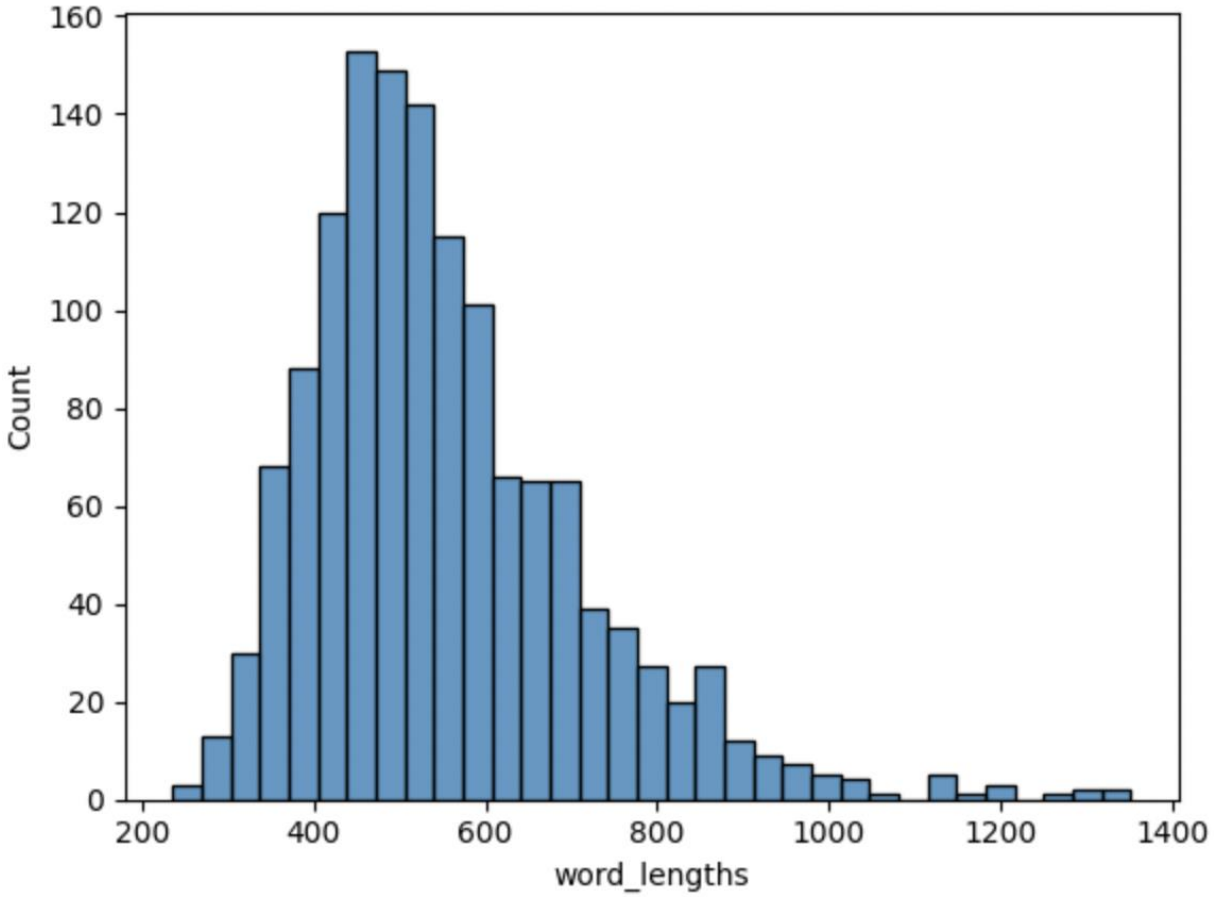
train_drcat_02.csv - https://www.kaggle.com/datasets/thedrcat/daigt-proper-train-dataset?select=train_drcat_02.csv

We combine the two datasets and mainly focus on two columns 'text' containing the actual human written or AI generated text and 'label' which is 0 for human written text and 1 for AI generated text. we combined 1300 AI generated texts from train_drcat_02.csv to train_essay.csv

We divide this combined dataset into train, validation, and test set in the ratio 70:15:15

EDA

I looked at the average word length of the text and plotted the histogram below. It can be seen that since these are essays the some of the texts are pretty long.



One thing we observed was that the train_essays.csv only contained 3 AI generated texts which is negligible for training any model. Hence, we combined 1300 AI generated texts from another data set train_drcat_02.csv

Approach

I treated this as a text classification problem and used three different models to solve it. First I used a multinomial Naive Bayes classifier, secondly I used LSTM and thirdly I used a pre-trained BERT model.

Multinomial Naive Bayes Classifier

For this approach first I clean the pre-process text data by Replacing newline characters (\n) with a space before and after, adding spaces around punctuation characters (/, !, ?, and .) when they appear between alphabetic characters, replacing repeating characters (more than 3 times) with a maximum length of 3, adding spaces around repeating characters (*, !, ?, and '), removing consecutive spaces (more than 2) and stripping leading/trailing spaces. I then use the

CountVectorizer to convert the text into a bag of words representation and then the TfidfTransformer to convert the bag of words representation to a TF-IDF (Term Frequency-Inverse Document Frequency) representation. I then feed it to a Multinomial NB classifier(you can use any other classifier as well). Below is the classification report of my model.

	precision	recall	f1-score	support
0	0.93	1.00	0.97	209
1	1.00	0.92	0.96	193
accuracy			0.96	402
macro avg	0.97	0.96	0.96	402
weighted avg	0.97	0.96	0.96	402

It gives an average accuracy of 96% on the test data.

LSTM

For the LSTM based approach I use a pre-trained word embedding model named GloVe (Global Vectors for Word Representation) with 50-dimensional vectors (glove.6B.50d). I use this to create the word embeddings for each token in a text. I use the nltk RegexpTokenizer and WordNetLemmatizer to tokenize the text and bring the tokens to their base form. I choose a sequence length for my sentences and pad if its short or truncate if its longer. I pass these embeddings to my LSTM model. I train my model for 20 epochs.

```
Epoch 1/20
59/59 [==============================] - 30s 325ms/step - loss: 0.7361 - accuracy: 0.8810 - auc:
0.9391 - val_loss: 0.2340 - val_accuracy: 0.9403 - val_auc: 0.9719
Epoch 2/20
59/59 [==============================] - 16s 269ms/step - loss: 0.2570 - accuracy: 0.9589 - auc:
0.9877 - val_loss: 0.0501 - val_accuracy: 0.9851 - val_auc: 0.9990
Epoch 3/20
59/59 [==============================] - 8s 126ms/step - loss: 0.1446 - accuracy: 0.9723 - auc:
0.9964 - val_loss: 0.0605 - val_accuracy: 0.9776 - val_auc: 0.9991
Epoch 4/20
59/59 [==============================] - 7s 116ms/step - loss: 0.1140 - accuracy: 0.9787 - auc:
0.9975 - val_loss: 0.0515 - val_accuracy: 0.9826 - val_auc: 0.9993
Epoch 5/20
59/59 [==============================] - 16s 279ms/step - loss: 0.0996 - accuracy: 0.9813 - auc:
0.9981 - val_loss: 0.0348 - val_accuracy: 0.9851 - val_auc: 0.9995
Epoch 6/20
59/59 [==============================] - 7s 125ms/step - loss: 0.0886 - accuracy: 0.9840 - auc:
0.9983 - val_loss: 0.0533 - val_accuracy: 0.9851 - val_auc: 0.9970
Epoch 7/20
```

```
59/59 [==============================] - 15s 263ms/step - loss: 0.0723 - accuracy: 0.9899 - auc:
0.9987 - val_loss: 0.0307 - val_accuracy: 0.9900 - val_auc: 0.9993
Epoch 8/20
59/59 [==============================] - 24s 409ms/step - loss: 0.0560 - accuracy: 0.9899 - auc:
0.9995 - val_loss: 0.0289 - val_accuracy: 0.9950 - val_auc: 0.9992
Epoch 9/20
59/59 [==============================] - 7s 117ms/step - loss: 0.0527 - accuracy: 0.9920 - auc:
0.9991 - val_loss: 0.0368 - val_accuracy: 0.9900 - val_auc: 0.9967
Epoch 10/20
59/59 [==============================] - 16s 269ms/step - loss: 0.0539 - accuracy: 0.9893 - auc:
0.9995 - val_loss: 0.0199 - val_accuracy: 0.9950 - val_auc: 0.9997
Epoch 11/20
59/59 [==============================] - 7s 121ms/step - loss: 0.0457 - accuracy: 0.9931 - auc:
0.9991 - val_loss: 0.0312 - val_accuracy: 0.9950 - val_auc: 0.9971
Epoch 12/20
59/59 [==============================] - 8s 134ms/step - loss: 0.0696 - accuracy: 0.9867 - auc:
0.9988 - val_loss: 0.0342 - val_accuracy: 0.9950 - val_auc: 0.9968
Epoch 13/20
59/59 [==============================] - 7s 113ms/step - loss: 0.0406 - accuracy: 0.9936 - auc:
0.9992 - val_loss: 0.0253 - val_accuracy: 0.9950 - val_auc: 0.9995
Epoch 14/20
59/59 [==============================] - 7s 125ms/step - loss: 0.0351 - accuracy: 0.9947 - auc:
0.9993 - val_loss: 0.0328 - val_accuracy: 0.9925 - val_auc: 0.9970
Epoch 15/20
59/59 [==============================] - 16s 281ms/step - loss: 0.0356 - accuracy: 0.9947 - auc:
0.9993 - val_loss: 0.0165 - val_accuracy: 0.9950 - val_auc: 0.9998
Epoch 16/20
59/59 [==============================] - 8s 127ms/step - loss: 0.0293 - accuracy: 0.9947 - auc:
0.9999 - val_loss: 0.0303 - val_accuracy: 0.9950 - val_auc: 0.9972
Epoch 17/20
59/59 [==============================] - 7s 114ms/step - loss: 0.0407 - accuracy: 0.9920 - auc:
0.9992 - val_loss: 0.0285 - val_accuracy: 0.9950 - val_auc: 0.9970
Epoch 18/20
59/59 [==============================] - 7s 127ms/step - loss: 0.0314 - accuracy: 0.9957 - auc:
0.9999 - val_loss: 0.0630 - val_accuracy: 0.9751 - val_auc: 0.9964
Epoch 19/20
59/59 [==============================] - 8s 132ms/step - loss: 0.0265 - accuracy: 0.9947 - auc:
0.9999 - val_loss: 0.0466 - val_accuracy: 0.9950 - val_auc: 0.9945
Epoch 20/20
59/59 [==============================] - 11s 180ms/step - loss: 0.0228 - accuracy: 0.9952 - auc:
0.9999 - val_loss: 0.0238 - val_accuracy: 0.9950 - val_auc: 0.9996
<keras.src.callbacks.History at 0x7899f84a2860>
```

Below is the classification report of my LSTM model.

	precision	recall	f1-score	support
0	0.99	1.00	0.99	209
1	0.99	0.98	0.99	193
accuracy			0.99	402
macro avg	0.99	0.99	0.99	402
weighted avg	0.99	0.99	0.99	402

The accuracy is about 99%. But I think if we test it on different or bigger dataset this will drop. I believe there is some amount of overfitting. But overall, our LSTM model performs better.

BERT

BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained natural language processing (NLP) model introduced by Google in 2018. It belongs to the transformer architecture family. Working with BERT is easier as it comes with its own tokenizer which takes care of all the text pre-processing. I adding the special tokens like [CLS] (classification token) and [SEP] (separator token) and fixed the max sequence length to 128. This automatically truncated anything longer or pads anything shorter. This has a disadvantage as well as we lose context. I use the bert-based-uncased model as it is comparatively smaller and faster to fine-tune. I train the model for 10 epochs.

```
Epoch 1/10, Validation Loss: 0.043367796750994870
Epoch 2/10, Validation Loss: 0.014970916026618843
Epoch 3/10, Validation Loss: 0.018481530153247362
Epoch 4/10, Validation Loss: 0.08556239033236589
Epoch 5/10, Validation Loss: 0.04473436697427293
Epoch 6/10, Validation Loss: 0.045099862596115974
Epoch 7/10, Validation Loss: 0.04660329660777386
Epoch 8/10, Validation Loss: 0.04679747709237477
Epoch 9/10, Validation Loss: 0.04725598137769802
Epoch 10/10, Validation Loss: 0.049738177483816925
```

Below is the classification report of the model on the test set.

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       209
           1       1.00      1.00      1.00       193

    accuracy                           1.00       402
   macro avg       1.00      1.00      1.00       402
weighted avg       1.00      1.00      1.00       402
```

This is highly biased and pretty sure a different dataset will yield much lower result as it intuitively is overfitting a certain distribution of data and might not generalize well.

NOTES

The codes submitted are ipynb kernels. The data is provided in zip files. So are the models. For the Bert based uncased pre-trained model download the files from https://www.kaggle.com/datasets/abhishek/bert-base-uncased and keep them in a folder bert-based-uncased. You should import the transformer package if not there you need to install using pip install. For test keep the fine-tuned models in the folder named in the code.